SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

Design and Implementation of a Database Management System for Library Circulation and Digital Resource Management

Program	AI/ML
Section	AI/ML RHINOS
Faculty	Miss Sahiti
Date of Submission	15/10/2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102103	Jai Havish Sannapareddy	[Enter mobile]	[Enter email]

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Miss Sahiti, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **“Design and Implementation of a Database Management System for Library Circulation and Digital Resource Management”**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date :

Place :

Name: Jai Havish Sannapareddy
Roll Number: 25WU0102103
Program: AI/ML

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. LOGICAL DATABASE DESIGN	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	13
2.3 Functional Dependencies	15
2.4 Normalization up to Third Normal Form (3NF)	16
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	19
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	22
3.1 Application Overview	22
3.2 Development Environment and Database Connectivity	22
3.3 Application Modules and Database Mapping	22
3.4 CRUD Operations	23
3.5 Business Rules, Validation and Error Handling	23
3.6 Reports and Database Operations	24
3.7 Application Screens	24
3.8 Testing and Results	30
4. CONCLUSION	32
4.1 Limitations and Future Enhancements	32
5. REFERENCES	33
6. APPENDICES	34

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	10
Table 4	Entity and Relationship Summary	12
Table 5	Relational Schema	13
Table 6	Keys and Integrity Constraints	13
Table 7	Normalization Status up to 3NF	16
Table 8	Data Dictionary	17
Table 9	Database Implementation Environment	19
Table 10	Application Development Environment	22
Table 11	Application Module-to-Database Mapping	22
Table 12	CRUD Operations	23
Table 13	Business Rules, Validation and Error Handling	23
Table 14	Reports and Database Operations	24
Table 15	Test Cases	30

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	11
Figure 2	Dashboard with key figures, overdue copies and popular books	25
Figure 3	Catalogue search for “database”	25
Figure 4	Member list with borrowing history of the selected member	26
Figure 5	Issuing a book to a member (status bar shows the due date)	26
Figure 6	Returning an overdue book creates a fine	27
Figure 7	Reservation queue for Wings of Fire	27
Figure 8	Unpaid fines, including the fine created by the return	28
Figure 9	Report: overdue copies with fine so far	28
Figure 10	Report: fine collection by month	29
Figure 11	Digital resources and usage	29

LIST OF ACRONYMS

Acronym	Expansion
3NF	Third Normal Form
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
ISBN	International Standard Book Number
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language

ABSTRACT

Many college libraries still track books in paper registers or loose spreadsheets. In such a setup it is hard to know which copy is on the shelf, who is keeping a book past its due date, or how much fine a member owes. This project builds a relational database and a small desktop application to remove those problems. The aim was to design a database for a library that handles books, digital resources, members, issues, returns, reservations and fines, and to put a simple front end on it. The design started from an ER model with fifteen entities, which was converted into a relational schema in Third Normal Form and built in MySQL 8.0 with keys and UNIQUE, NOT NULL, CHECK and DEFAULT constraints. The front end is written in Python with Tkinter. It supports catalogue maintenance, member registration, issue, return, reservation, fine payment, search and seven reports. Rules such as one active issue per copy are enforced by the database itself, and multi-step actions run inside transactions. Forty automated tests, valid and invalid cases, pass on the final system. The result is a tested library system that a small library could use for daily circulation.

KEYWORDS

Keywords: Library Management, Relational Database, ER Diagram, Normalization, MySQL, Python

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A small library keeps three kinds of things: books, the people who borrow them, and the record of who borrowed what. When this record lives in a register, simple questions become slow to answer. A librarian has to flip through pages to find out whether a title is on the shelf. Overdue books are noticed only when someone happens to look at the due dates. Fines are worked out by hand, so mistakes and arguments are common. When a popular book is out, there is no fair way to remember who asked for it first.

The library staff, the members and the administrators are all affected. Staff lose time on routine lookups. Members do not know when a book will be free. Administrators cannot easily see which books are popular or how much fine money is pending. Digital resources such as e-books and journals make it worse, because their use is not recorded anywhere. A shared database that checks every entry and a simple application on top of it will replace the registers and give the same answer to everybody.

1.2 Objectives

- Design an ER model and a normalised relational schema (up to 3NF) for books, editions, copies, members, issues, reservations, fines and digital resources.
- Implement the schema in MySQL with keys and constraints that enforce rules such as unique ISBN and accession numbers, one active issue per copy and non-negative fines.
- Develop a Python application that performs catalogue maintenance, member registration, issue, return, reservation, fine collection and search.
- Generate circulation reports (available titles, overdue copies, member history, popular books, reservations and fine collection) using joins, aggregates, nested queries and views.
- Validate the system with positive and negative test cases and show that transactions leave no half-finished data.

1.3 Scope and Assumptions

Table 1 states what the project covers and what it leaves out.

Table 1. Scope and Assumptions

Item	Description
In Scope	Catalogue of titles, editions, authors, publishers and categories; physical copies with accession numbers; member registration for three member types; issue, return and renewal; reservation queue; fine calculation and payment; digital resource access log; search; seven reports; desktop application for library staff.
Out of Scope / Exclusions	Member self-service login, online payment, barcode scanners, e-mail or SMS reminders, lost-book charges, fine waivers, purchase and acquisition of new stock, and storing the digital files themselves.
Assumptions	The librarian operates the program on behalf of members at the desk. One library branch only. Loan period and fine rate depend on member type (Student 14 days at Rs 2 per day, Faculty 30 days at Rs 1, Public 7 days at Rs 5). A member with more than Rs 50 unpaid fine cannot borrow. Fine is charged when the book is returned.

1.4 Proposed Solution Strategy

The solution keeps all rules in one place. The database refuses data that breaks a rule, and the application adds the rules that need several rows to be checked together. Table 2 summarises the approach. The relational model [1] is used for the data, so every fact is stored once.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Librarian (daily circulation and catalogue work); Administrator (reports and policy); Member (student, faculty or public, served at the desk).
Major Application Modules	Dashboard, Catalogue, Members, Issue & Return, Reservations, Fines, Digital Resources, Reports.
Proposed Workflow	A member is registered once. The librarian issues a copy by picking the member and the copy. On return the system closes the loan, adds a fine if the book is late, and either puts the copy back on the shelf or holds it for the first member in the reservation queue.
Important Business Rules	Unique ISBN and accession number; one active issue per copy; return date not before issue date; book limit by member type; reservation queue served in order; fines never negative and never over-paid.
Expected Reports / Outputs	Available titles, overdue copies with fine so far, member borrowing history, popular books, reservation queue, fine collection by month, digital resource usage.

1.5 Functional Requirements

The main functions the system must provide are listed in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Add and search titles, editions, authors, publishers and categories	High
FR-02	Add physical copies with a unique accession number and withdraw copies without losing history	High
FR-03	Register members with type, card number, e-mail, phone and expiry date	High

FR-04	Issue a copy to a member after checking membership, book limit, fines and copy status	High
FR-05	Return a copy, close the loan and calculate the fine for late return	High
FR-06	Renew a loan when it is not overdue and nobody is waiting	Medium
FR-07	Reserve a title that has no copy on the shelf and keep a fair queue	High
FR-08	Hold a returned copy for the first person in the queue	High
FR-09	Show fines per member and record full or part payment	High
FR-10	Record use of digital resources and refuse use after the licence ends	Medium
FR-11	Search the catalogue by title, author, category or ISBN	High
FR-12	Produce circulation reports and save them as CSV	Medium

1.6 Entity-Relationship Diagram (ERD)

The approved ER diagram is shown in Figure 1 [2]. It has fifteen entities. The cardinality of each relationship is written next to the lines. PK marks a primary key and FK a foreign key.

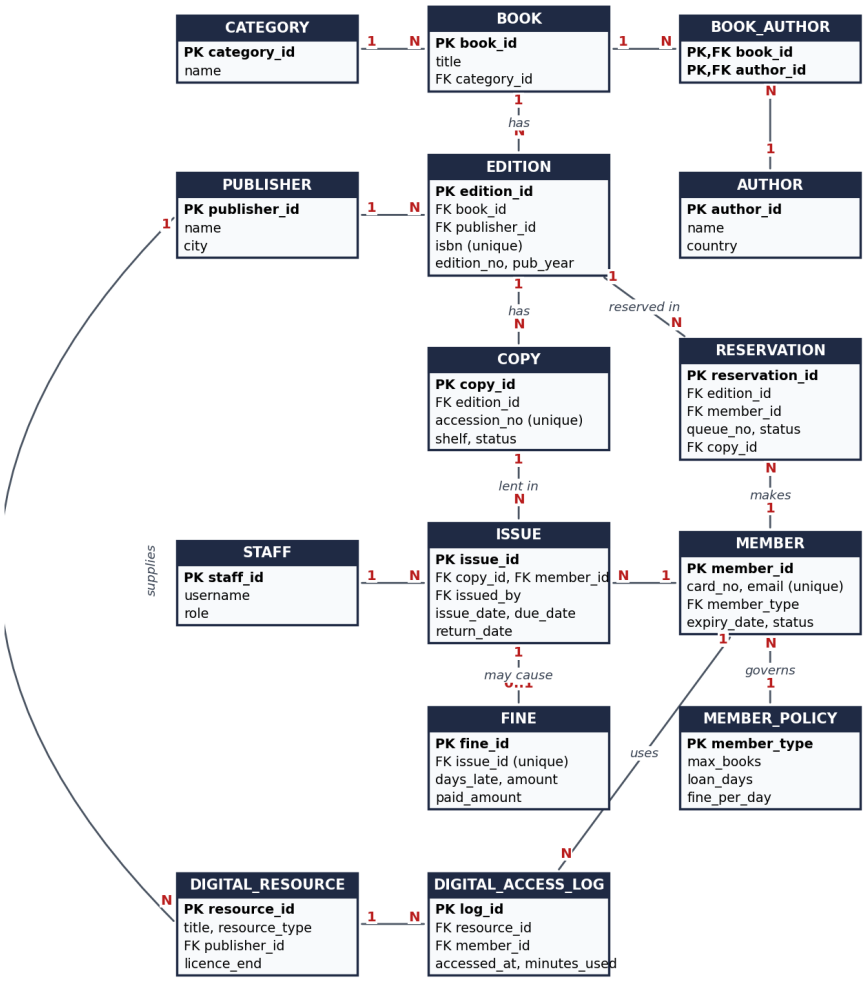


Figure 1. Entity-Relationship Diagram of the Proposed System

A title (BOOK) is separate from its EDITION and from each physical COPY. This avoids repeating the title and publisher on every copy row, and it lets the library own several editions of the same title.

1.7 Entity and Relationship Summary

Table 4 summarises the entities and their relationships.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
PUBLISHER	publisher_id, name, city. 1 publisher : N editions	Who publishes an edition; also supplies digital resources
AUTHOR	author_id, name, country. M:N with BOOK	People who write the titles
CATEGORY	category_id, name. 1 category : N books	Subject grouping used in search
BOOK	book_id, title, category_id. 1 book : N editions	One row per title
BOOK_AUTHOR	book_id, author_id (composite key)	Resolves the many-to-many link between books and authors
EDITION	edition_id, isbn, edition_no, pub_year, pages. 1 edition : N copies	One row per printed edition; ISBN is unique
COPY	copy_id, accession_no, shelf, status. 1 copy : N issues over time	Physical item; status is AVAILABLE, ISSUED, ON_HOLD, LOST or WITHDRAWN
MEMBER_POLICY	member_type, max_books, loan_days, fine_per_day. 1 type : N members	Loan rules that depend only on member type
MEMBER	member_id, card_no, name, email, phone, expiry_date, status	A registered library user
STAFF	staff_id, username, role. 1 staff : N issues	Librarian or admin who records the issue
ISSUE	issue_id, issue_date, due_date, return_date. 1 issue : 0..1 fine	One loan of one copy to one member
RESERVATION	reservation_id, queue_no, status, copy_id. N per edition	A member's place in the waiting line for a title
FINE	fine_id, days_late, amount, paid_amount	Charge for a late return, linked to the issue
DIGITAL_RESOURCE	resource_id, title, resource_type, url, licence_end	E-book, journal, video or database the library subscribes to
DIGITAL_ACCESS_LOG	log_id, accessed_at, minutes_used. Links MEMBER and DIGITAL_RESOURCE (M:N)	Records each use of a digital resource

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was converted into the 15 relations in Table 5. Each entity became a table, and the many-to-many links became tables of their own (BOOK_AUTHOR and DIGITAL_ACCESS_LOG).

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	publisher	Publishers of editions and digital resources
2	author	Authors of titles
3	category	Subject categories
4	member_policy	Loan days, book limit and fine rate for each member type
5	staff	Library staff who operate the system
6	book	Titles
7	book_author	Links titles to their authors
8	edition	Printed editions with ISBN
9	copy	Physical copies with accession number and status
10	member	Library members
11	issue	Loans of copies to members
12	reservation	Reservation queue for editions
13	fine	Fines raised on late returns
14	digital_resource	Digital resources and licence end dates
15	digital_access_log	Use of digital resources by members

2.2 Keys and Integrity Constraints

Table 6 lists the keys and constraints. Every foreign key points to a parent table, so no row can refer to something that does not exist.

Table 6. Keys and Integrity Constraints

digital_access_log	Key / Constraint	Attribute(s)	Business Rule Enforced
	Primary Key	publisher_id	Identifies a publisher
	Unique / Not Null	name	No duplicate publisher names
	Primary Key / Not Null	author_id / name	Every author has a name

digital_access_log	Key / Constraint	Attribute(s)	Business Rule Enforced
	Primary Key / Unique	category_id / name	No duplicate categories
	Primary Key	member_type	One rule set per type
	Check	max_books > 0, loan_days > 0, fine_per_day >= 0	Rules must be sensible
	Primary Key / Foreign Key	book_id / category_id	Each title belongs to a category
	Composite Primary Key	book_id, author_id	A pair appears only once
	Foreign Key	book_id (ON DELETE CASCADE), author_id	No link without both parents
	Primary Key / Unique	edition_id / isbn	ISBN is unique
	Unique	book_id, publisher_id, edition_no	Same edition cannot be added twice
	Foreign Key	book_id, publisher_id	Edition needs a title and a publisher
	Check	isbn REGEXP '^[0-9]{13}$'; edition_no > 0; pub_year 1450-2100; pages > 0	Valid ISBN and dates
	Primary Key / Unique	copy_id / accession_no	Accession number is unique
	Foreign Key	edition_id	A copy belongs to an edition
	Default / Not Null	status = 'AVAILABLE'; added_on = today	New copies start on the shelf
	Primary Key / Unique	member_id / card_no, email	No duplicate card or e-mail
	Foreign Key	member_type	Type must exist in member_policy
	Check	expiry_date > joined_on; email LIKE '%_@_%._%'	Valid dates and e-mail
	Default	member_type = 'STUDENT'; status = 'ACTIVE'	Common case needs no input
	Primary Key / Foreign Key	issue_id / copy_id, member_id, issued_by	Loan links copy, member and staff
	Unique (generated column)	active_copy_id = IF(return_date IS NULL, copy_id, NULL)	One active issue per copy
	Check	due_date > issue_date; return_date IS NULL OR return_date >= issue_date	Valid issue and return dates
	Primary Key / Foreign Key	reservation_id / edition_id, member_id, copy_id	Reservation links edition and member
	Unique (generated columns)	(edition_id, live_queue_no); (edition_id, live_member_id)	Queue order is unique; one live reservation per member per title
	Check	queue_no > 0; status <> 'READY' OR copy_id IS NOT NULL	A ready reservation always has a copy
	Primary Key / Unique	fine_id / issue_id	At most one fine per loan
	Check	days_late >= 0; amount >= 0; paid_amount >= 0; paid_amount <= amount	Non-negative and never over-paid
	Primary Key / Check by ENUM	resource_id / resource_type	Only EBOOK, JOURNAL, VIDEO or DATABASE

digital_access_log	Key / Constraint	Attribute(s)	Business Rule Enforced
	Primary Key / Foreign Key	log_id / resource_id, member_id	Each use is tied to a member and a resource
	Check / Default	minutes_used >= 0; accessed_at = now	No negative time

Referential actions: foreign keys use the default RESTRICT, so a title, member or copy that has history cannot be deleted. Only book_author uses ON DELETE CASCADE. The program withdraws copies and cancels reservations by changing status instead of deleting rows.

2.3 Functional Dependencies

The important functional dependencies are given below. Each determinant on the left is a candidate key of its table, so the design also satisfies Boyce-Codd Normal Form.

- publisher: publisher_id → name, city; name → publisher_id
- author: author_id → name, country
- category: category_id → name; name → category_id
- member_policy: member_type → max_books, loan_days, fine_per_day
- book: book_id → title, category_id
- edition: edition_id → book_id, publisher_id, isbn, edition_no, pub_year, pages; isbn → edition_id; (book_id, publisher_id, edition_no) → edition_id
- copy: copy_id → edition_id, accession_no, shelf, status, added_on; accession_no → copy_id
- member: member_id → card_no, full_name, email, phone, member_type, joined_on, expiry_date, status; card_no → member_id; email → member_id
- issue: issue_id → copy_id, member_id, issued_by, issue_date, due_date, return_date
- reservation: reservation_id → edition_id, member_id, queue_no, reserved_on, status, copy_id, ready_on
- fine: fine_id → issue_id, days_late, amount, paid_amount, assessed_on, paid_on; issue_id → fine_id
- digital_resource: resource_id → title, resource_type, publisher_id, url, licence_end
- digital_access_log: log_id → resource_id, member_id, accessed_at, minutes_used

2.4 Normalization

Normalization was done on the single sheet a librarian would keep by hand, with one wide row for every loan. Table 7 shows that every final relation satisfies each normal form [3], [4].

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
publisher	Yes	Yes	Yes	Yes	Already in 3NF
author	Yes	Yes	Yes	Yes	Already in 3NF
category	Yes	Yes	Yes	Yes	Already in 3NF
member_policy	Yes	Yes	Yes	Yes	Split from MEMBER (type → loan rules)
staff	Yes	Yes	Yes	Yes	Already in 3NF
book	Yes	Yes	Yes	Yes	Split from the loan sheet (title, category)
book_author	Yes	Yes	Yes	Yes	Created to remove author list (1NF)
edition	Yes	Yes	Yes	Yes	ISBN, year, publisher moved out of COPY
copy	Yes	Yes	Yes	Yes	Physical item only; no title data
member	Yes	Yes	Yes	Yes	Person data separated from loan data
issue	Yes	Yes	Yes	Yes	Only loan facts remain
reservation	Yes	Yes	Yes	Yes	Queue facts only
fine	Yes	Yes	Yes	Yes	Member found through ISSUE, so not repeated
digital_resource	Yes	Yes	Yes	Yes	Already in 3NF
digital_access_log	Yes	Yes	Yes	Yes	Link table for M:N use

1NF: The sheet stored a list of authors in one cell. Authors were moved to their own table and linked through BOOK_AUTHOR, so every cell holds one value and every table has a primary key.

2NF: Member name depended only on the card number and title depended only on the copy, not on the whole loan. These facts moved to MEMBER, COPY, EDITION and BOOK, so ISSUE keeps only facts about the loan.

3NF: member_type determined max_books, loan_days and fine_per_day, so these went to MEMBER_POLICY. Category and publisher details were separated in the same way. FINE does not store member_id, because issue_id already gives the member.

Two columns look like repeated data but are not. reservation.copy_id is filled only while a copy is being held for that reservation. issue.active_copy_id and the live_* columns of RESERVATION are generated by MySQL for constraints, and nobody types them in.

2.5 Data Dictionary

Table 8 is the data dictionary. It was generated from the live database, so the data types and keys match what MySQL actually holds.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
publisher	publisher_id	INT	PK, AUTO_INCREMENT	Unique publisher number
	name	VARCHAR(100)	UNIQUE, NOT NULL	Publisher name
	city	VARCHAR(60)		City of the publisher
author	author_id	INT	PK, AUTO_INCREMENT	Unique author number
	name	VARCHAR(100)	NOT NULL	Author's full name
	country	VARCHAR(60)		Author's country
category	category_id	INT	PK, AUTO_INCREMENT	Unique category number
	name	VARCHAR(60)	UNIQUE, NOT NULL	Category name
member_policy	member_type	VARCHAR(10)	PK	Member type: STUDENT, FACULTY or PUBLIC
	max_books	TINYINT	NOT NULL	Most books held at one time
	loan_days	TINYINT	NOT NULL	Days a book may be kept
	fine_per_day	DECIMAL(5,2)	NOT NULL, DEFAULT 2.00	Fine for each late day (Rs)
staff	staff_id	INT	PK, AUTO_INCREMENT	Unique staff number
	username	VARCHAR(30)	UNIQUE, NOT NULL	Login name
	full_name	VARCHAR(80)	NOT NULL	Staff member's name
	role	ENUM	NOT NULL, DEFAULT LIBRARIAN	LIBRARIAN or ADMIN
book	book_id	INT	PK, AUTO_INCREMENT	Unique title number
	title	VARCHAR(200)	NOT NULL	Title of the book
	category_id	INT	FK, NOT NULL	Category of the title
book_author	book_id	INT	PK, FK	Title
	author_id	INT	PK, FK	Author of that title
edition	edition_id	INT	PK, AUTO_INCREMENT	Unique edition number
	book_id	INT	FK, NOT NULL	Title this edition belongs to
	publisher_id	INT	FK, NOT NULL	Publisher of the edition

Table	Attribute	Data Type	Key / Constraint	Description
	isbn	CHAR(13)	UNIQUE, NOT NULL	13-digit ISBN
	edition_no	TINYINT	NOT NULL, DEFAULT 1	Edition number
	pub_year	SMALLINT	NOT NULL	Year of publication
	pages	SMALLINT		Number of pages
copy	copy_id	INT	PK, AUTO_INCREMENT	Unique copy number
	edition_id	INT	FK, NOT NULL	Edition this copy belongs to
	accession_no	VARCHAR(15)	UNIQUE, NOT NULL	Library's own number for the copy
	shelf	VARCHAR(20)		Shelf location
	status	ENUM	NOT NULL, DEFAULT AVAILABLE	Where the copy is now (AVAILABLE, ISSUED, ON_HOLD, LOST, WITHDRAWN)
	added_on	DATE	GENERATED	Date the copy was added
member	member_id	INT	PK, AUTO_INCREMENT	Unique member number
	card_no	VARCHAR(12)	UNIQUE, NOT NULL	Library card number
	full_name	VARCHAR(100)	NOT NULL	Member's name
	email	VARCHAR(120)	UNIQUE, NOT NULL	E-mail address
	phone	VARCHAR(15)		Phone number
	member_type	VARCHAR(10)	FK, NOT NULL, DEFAULT STUDENT	Type; decides loan rules
	joined_on	DATE	GENERATED	Date of joining
	expiry_date	DATE	NOT NULL	Date membership ends
	status	ENUM	NOT NULL, DEFAULT ACTIVE	ACTIVE, SUSPENDED or EXPIRED
issue	issue_id	INT	PK, AUTO_INCREMENT	Unique loan number
	copy_id	INT	FK, NOT NULL	Copy lent
	member_id	INT	FK, NOT NULL	Borrower
	issued_by	INT	FK	Staff member who issued it
	issue_date	DATE	NOT NULL	Date of issue
	due_date	DATE	NOT NULL	Date due back
	return_date	DATE		Date returned; NULL while on loan
	active_copy_id	INT	UNIQUE, GENERATED	Generated: copy_id while on loan, else NULL
reservation	reservation_id	INT	PK, AUTO_INCREMENT	Unique reservation number
	edition_id	INT	FK, NOT NULL	Title reserved

Table	Attribute	Data Type	Key / Constraint	Description
	member_id	INT	FK, NOT NULL	Member waiting
	queue_no	INT	NOT NULL	Place in line; keeps counting up
	reserved_on	DATETIME	GENERATED	Date and time of reservation
	status	ENUM	NOT NULL, DEFAULT WAITING	WAITING, READY, FULFILLED, CANCELLED or EXPIRED
	copy_id	INT	FK	Copy held for the member once READY
	ready_on	DATE		Date the copy was set aside
	live_queue_no	INT	GENERATED	Generated: queue_no while reservation is live
	live_member_id	INT	GENERATED	Generated: member_id while reservation is live
fine	fine_id	INT	PK, AUTO_INCREMENT	Unique fine number
	issue_id	INT	FK, UNIQUE, NOT NULL	Late loan the fine belongs to
	days_late	SMALLINT	NOT NULL	Days past the due date
	amount	DECIMAL(8,2)	NOT NULL, DEFAULT 0.00	Fine amount in rupees
	paid_amount	DECIMAL(8,2)	NOT NULL, DEFAULT 0.00	Amount paid so far
	assessed_on	DATE	GENERATED	Date fine was raised
	paid_on	DATE		Date of latest payment
digital_resource	resource_id	INT	PK, AUTO_INCREMENT	Unique resource number
	title	VARCHAR(200)	NOT NULL	Title of the resource
	resource_type	ENUM	NOT NULL	EBOOK, JOURNAL, VIDEO or DATABASE
	publisher_id	INT	FK	Supplier or publisher
	url	VARCHAR(255)	NOT NULL	Address of the resource
	licence_end	DATE		Date the licence ends
digital_access_log	log_id	INT	PK, AUTO_INCREMENT	Unique log number
	resource_id	INT	FK, NOT NULL	Resource used
	member_id	INT	FK, NOT NULL	Member who used it
	accessed_at	DATETIME	GENERATED	Date and time of use
	minutes_used	SMALLINT	NOT NULL, DEFAULT 0	Minutes spent

2.6 Implementation of Logical Design: DDL and Sample Data

The database was built in MySQL [5]. The complete script is in Appendix A. Three representative CREATE TABLE statements follow: the ISSUE table, which enforces one active issue per copy, the RESERVATION table, which keeps the queue in order, and the FINE table.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL
DBMS Version	8.0.46
Number of Tables	15 tables and 6 views
Sample Data	About 210 rows: 15 titles, 16 editions, 36 copies, 12 members, 34 loans, 7 fines, 5 reservations, 6 digital resources and 24 access records

The statements below show the three most important tables. The remaining tables follow the same style.

```
CREATE TABLE issue (
    issue_id    INT AUTO_INCREMENT PRIMARY KEY,
    copy_id     INT  NOT NULL,
    member_id   INT  NOT NULL,
    issued_by   INT,
    issue_date  DATE NOT NULL,
    due_date    DATE NOT NULL,
    return_date DATE NULL,
    active_copy_id INT GENERATED ALWAYS AS (IF(return_date IS NULL, copy_id, NULL)) STORED,
    UNIQUE (active_copy_id),
    FOREIGN KEY (copy_id)   REFERENCES copy(copy_id),
    FOREIGN KEY (member_id) REFERENCES member(member_id),
    FOREIGN KEY (issued_by) REFERENCES staff(staff_id),
    CHECK (due_date > issue_date),
    CHECK (return_date IS NULL OR return_date >= issue_date)
);
```

```
CREATE TABLE reservation (
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    edition_id     INT NOT NULL,
    member_id      INT NOT NULL,
    queue_no       INT NOT NULL CHECK (queue_no > 0),
    reserved_on    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status         ENUM('WAITING','READY','FULFILLED','CANCELLED','EXPIRED')
                   NOT NULL DEFAULT 'WAITING',
    copy_id        INT NULL,
    ready_on       DATE NULL,
    live_queue_no  INT GENERATED ALWAYS AS
                   (IF(status IN ('WAITING','READY'), queue_no, NULL)) STORED,
    live_member_id INT GENERATED ALWAYS AS
                   (IF(status IN ('WAITING','READY'), member_id, NULL)) STORED,
    UNIQUE (edition_id, live_queue_no),
    UNIQUE (edition_id, live_member_id),
    FOREIGN KEY (edition_id) REFERENCES edition(edition_id),
    FOREIGN KEY (member_id)  REFERENCES member(member_id),
    FOREIGN KEY (copy_id)    REFERENCES copy(copy_id),
    CHECK (status <> 'READY' OR copy_id IS NOT NULL)
);
```

```
CREATE TABLE fine (
    fine_id     INT AUTO_INCREMENT PRIMARY KEY,
    issue_id    INT NOT NULL UNIQUE,
    days_late   SMALLINT     NOT NULL CHECK (days_late >= 0),
    amount      DECIMAL(8,2) NOT NULL DEFAULT 0 CHECK (amount >= 0),
    paid_amount DECIMAL(8,2) NOT NULL DEFAULT 0 CHECK (paid_amount >= 0),
    assessed_on DATE NOT NULL DEFAULT (CURRENT_DATE),
    paid_on     DATE NULL,
    FOREIGN KEY (issue_id) REFERENCES issue(issue_id),
    CHECK (paid_amount <= amount)
);
```

The sample data gives each table realistic rows. Dates are written as an offset from the current date, so the overdue and fine reports always have records to show. Row counts after loading the script are:

```
table                rows
-------------------------
publisher             6
author               18
category              6
member_policy         3
staff                 3
book                 15
book_author          19
edition              16
copy                 36
member               12
issue                34
reservation           5
fine                  7
digital_resource      6
digital_access_log   24
```

Sample rows from the member and copy tables:

```
card_no     full_name        member_type  expiry_date  status
LIB2025001  Aarav Sharma     STUDENT      +300 days    ACTIVE
LIB2024101  Dr. Neha Kapoor  FACULTY      +800 days    ACTIVE
LIB2025201  Manoj Kulkarni   PUBLIC       +165 days    ACTIVE
LIB2023015  Ishaan Joshi     STUDENT      -30 days     EXPIRED

accession_no  edition  shelf  status
ACC-0001      1        CS-A1  ISSUED
ACC-0002      1        CS-A1  AVAILABLE
ACC-0029      13       HI-E2  ON_HOLD
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is a desktop program for library staff. It opens on a dashboard that shows the key numbers, and a side menu leads to eight screens: Dashboard, Catalogue, Members, Issue & Return, Reservations, Fines, Digital and Reports. Nothing on the screens is typed in advance. Every list, drop-down and figure is read from the MySQL database when the screen opens, and every button writes back to it. All circulation rules live in one Python module, library.py, so the screens, the tests and any future interface use exactly the same checks.

3.2 Development Environment and Database Connectivity

The program connects through MySQL Connector/Python [7], with the host, user, password and database name read from environment variables. If the password is wrong the program asks for it. The interface uses Tkinter [6]. Table 10 gives the environment.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.13
DBMS	MySQL 8.0
IDE / Framework / GUI Toolkit	Tkinter with ttk (standard library); unittest for tests
Database Connector / Driver	mysql-connector-python
Operating Environment	Windows 11
Repository Link	Project folder submitted with this report (see Appendix B). Not published online.

3.3 Application Modules and Database Mapping

Table 11 shows which tables each screen reads or changes.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Show key numbers, overdue list and popular books	copy, issue, member, book, reservation, fine, digital_access_log, v_overdue_copies, v_popular_books	SELECT, COUNT, SUM
Catalogue	Search titles; add a book, add a copy, withdraw a copy	book, author, book_author, edition, publisher, category, copy	INSERT, SELECT, UPDATE (status)

Module / Screen	Purpose	Tables Used	Main Operations
Members	Register, search and show borrowing history	member, member_policy, v_member_history	INSERT, SELECT
Issue & Return	Issue, return and renew; fine at return	member, member_policy, copy, issue, fine, reservation	Transaction: INSERT, UPDATE
Reservations	Reserve, cancel and view the queue	reservation, edition, copy, member, v_reservation_queue	INSERT, UPDATE, SELECT
Fines	List fines and take payment	fine, issue, member, copy, edition, book	SELECT, UPDATE
Digital	Log access and show usage	digital_resource, digital_access_log, member	INSERT, SELECT, GROUP BY
Reports	Run seven reports and save CSV	Views and joins on all main tables	SELECT, JOIN, aggregate

3.4 CRUD Operations

Table 12 shows how the four basic operations appear in the application. The program never deletes loan history; it changes a status instead.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Add book, add copy, register member, issue book, reserve title, log digital access	book, author, edition, copy, member, issue, reservation, digital_access_log	Figures 5 and 7
Read / Search	Catalogue search, member history, dashboard, reports, fines list	All main tables and views	Figures 2, 3, 4, 9, 10
Update	Return book (sets return date and copy status, adds fine); renew; pay fine; promote reservation	issue, copy, fine, reservation	Figures 6 and 8
Delete / Controlled Removal	Withdraw copy (status WITHDRAWN); cancel reservation (status CANCELLED). No row is deleted, so history stays	copy, reservation	Figure 7 (Cancel selected)

3.5 Business Rules, Validation and Error Handling

Input is checked in Python before it reaches the database, and the database checks it again with its own constraints. All SQL uses parameter placeholders, so typed text can never become part of a query. Every action that changes more than one table runs in one transaction. If any step fails, the whole action is rolled back. Table 13 lists the main rules.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
ISBN has 13 digits	check_isbn() in Python and CHECK with REGEXP in the table	“ISBN must have exactly 13 digits.” (TC12)
ISBN and accession number are unique	Pre-check plus UNIQUE constraint	“This ISBN is already in the catalogue.” (TC11)
E-mail and phone are valid	Regular-expression checks; CHECK on e-mail column	“Email address is not valid.” (TC13)
Member must be active and not expired	Status and expiry date checked before issue or reserve	“Membership is expired.” (TC04)

Rule / Validation	How Implemented	Evidence / Message
Book limit by member type	Active loans counted and compared with member_policy.max_books	“Limit reached: Student members can hold 3 books.” (TC02)
Copy must be free	Row locked with SELECT … FOR UPDATE, then status checked	“This copy is not available (status: ISSUED).” (TC03)
One active issue per copy	UNIQUE index on generated column active_copy_id	Error 1062 Duplicate entry for key 'issue.active_copy_id' (TC16)
Held copy only for the right member	Reservation with status READY must match the member	“This copy is kept aside for another member.” (TC10)
Fine is correct and never negative or over-paid	days late × fine_per_day; CHECK constraints; payment compared with balance	“Amount is more than the balance of Rs 25.00.” (TC14); error 3819 (TC17)
Digital licence must be valid	licence_end compared with today	“The licence for this resource has ended.” (TC15)
All-or-nothing actions	Transaction wrapper commits on success and rolls back on any error	Copy stays AVAILABLE after refused issue (TC02)
No SQL injection	Parameterised queries (%s placeholders) everywhere	User text is never joined into SQL

3.6 Reports and Database Operations

The reports in Table 14 are run from the Reports screen and can be saved as CSV.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Available titles	View v_available_titles: JOIN, GROUP_CONCAT, COUNT(DISTINCT)	Title, edition, authors, copies on the shelf
Overdue copies	View v_overdue_copies: JOIN, DATEDIFF, arithmetic with fine_per_day	Copy, member, days overdue, fine so far (Figure 9)
Member history	View v_member_history: JOIN, LEFT JOIN, CASE	All loans of a member with status and fine (Figure 4)
Popular books	View v_popular_books: COUNT, GROUP BY, ORDER BY, LIMIT	Top titles by number of issues
Reservation queue	View v_reservation_queue: ROW_NUMBER() OVER (PARTITION BY …)	Position in line per title (Figure 7)
Fine collection by month	View v_fine_summary: SUM, GROUP BY DATE_FORMAT	Fined, collected and pending per month (Figure 10)
Books currently on loan	Multi-table JOIN with WHERE return_date IS NULL	Copy, member, issue and due dates
Digital resource usage	LEFT JOIN, GROUP BY, COALESCE	Visits and minutes per resource (Figure 11)
Titles never issued (SQL file)	Nested query: NOT IN (subquery)	List of unused titles
Unpaid fines above average (SQL file)	Subquery inside HAVING	Members owing more than the average unpaid fine

3.7 Application Screens

Figures 2 to 11 show the application running on the sample data. They cover search, data entry, a transaction, a rejected action and several reports.

The dashboard in Figure 2 is the first screen. Its numbers are counted live from the database.

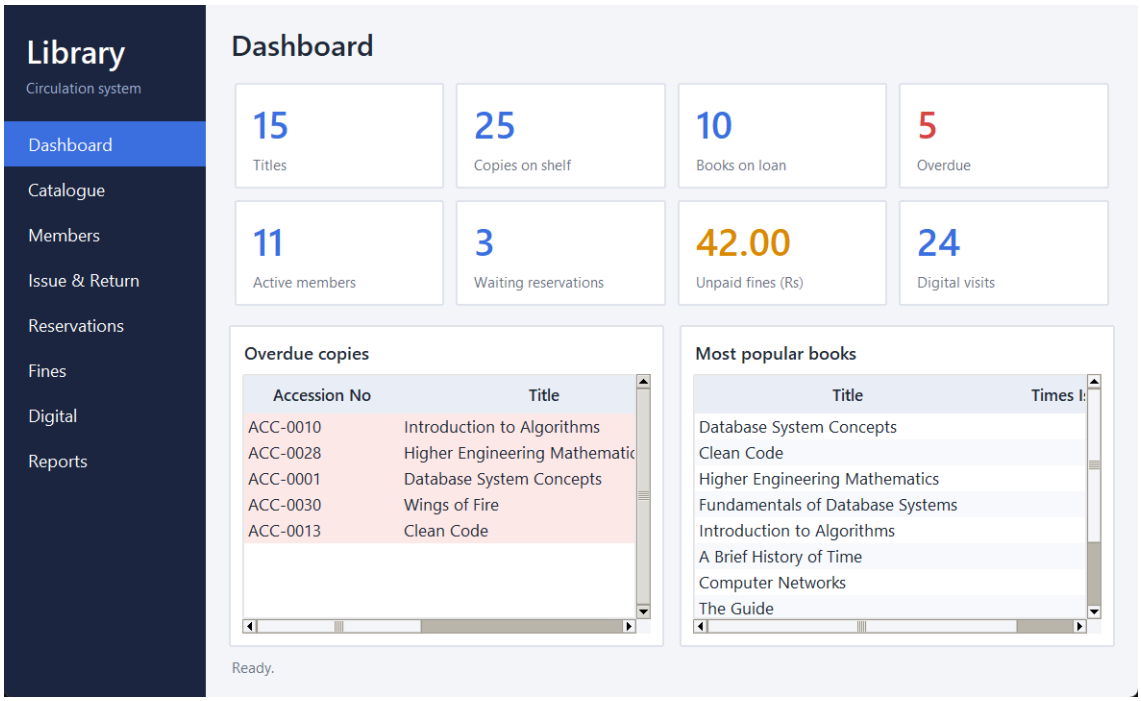


Figure 2. Dashboard with key figures, overdue copies and popular books

Figure 3 shows the catalogue after typing a search word. Rows are filtered as the user types.

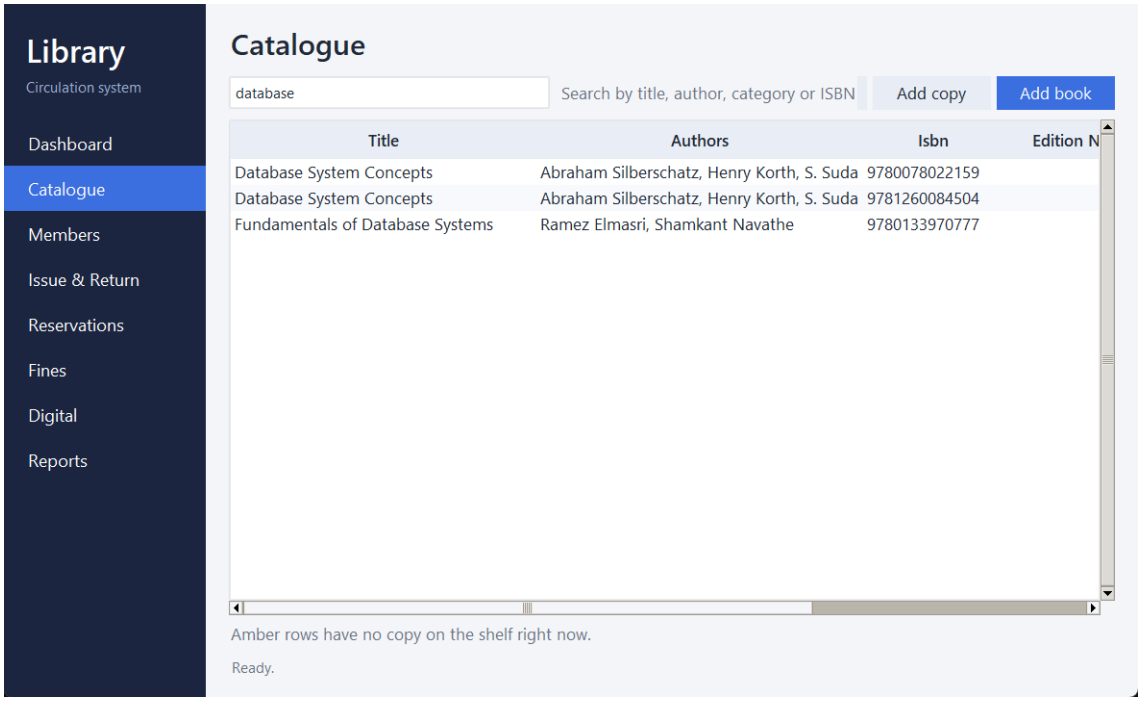


Figure 3. Catalogue search for “database”

Figure 4 shows the member list. Clicking a member shows that member's loans below. Red rows are overdue or expired.

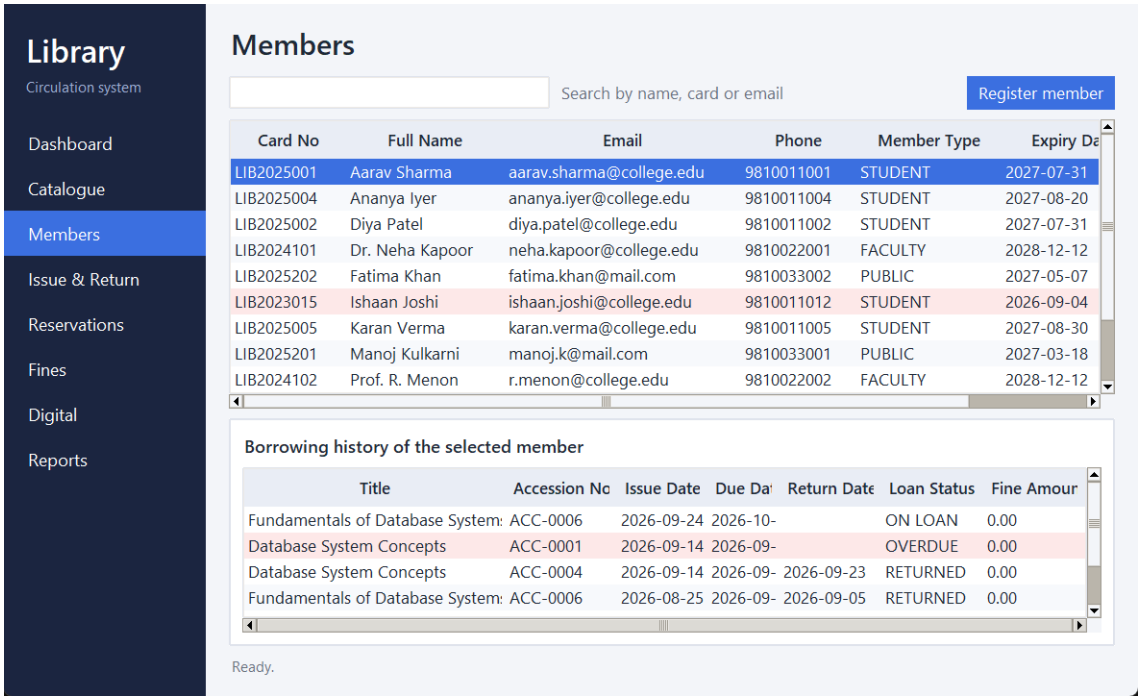


Figure 4. Member list with borrowing history of the selected member

Figure 5 shows a data-entry transaction. A copy was issued to a student, and the status bar reports the due date.

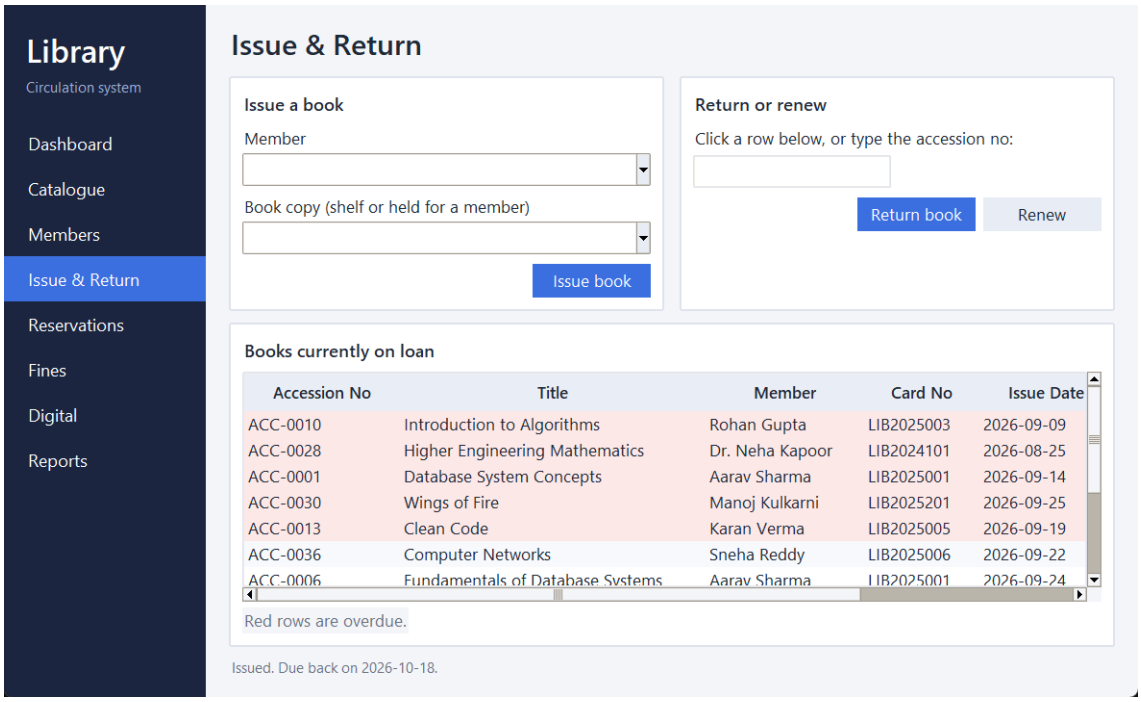


Figure 5. Issuing a book to a member (status bar shows the due date)

Figure 6 shows the return of an overdue copy. The status bar shows the fine, and the loan list no longer holds the copy.

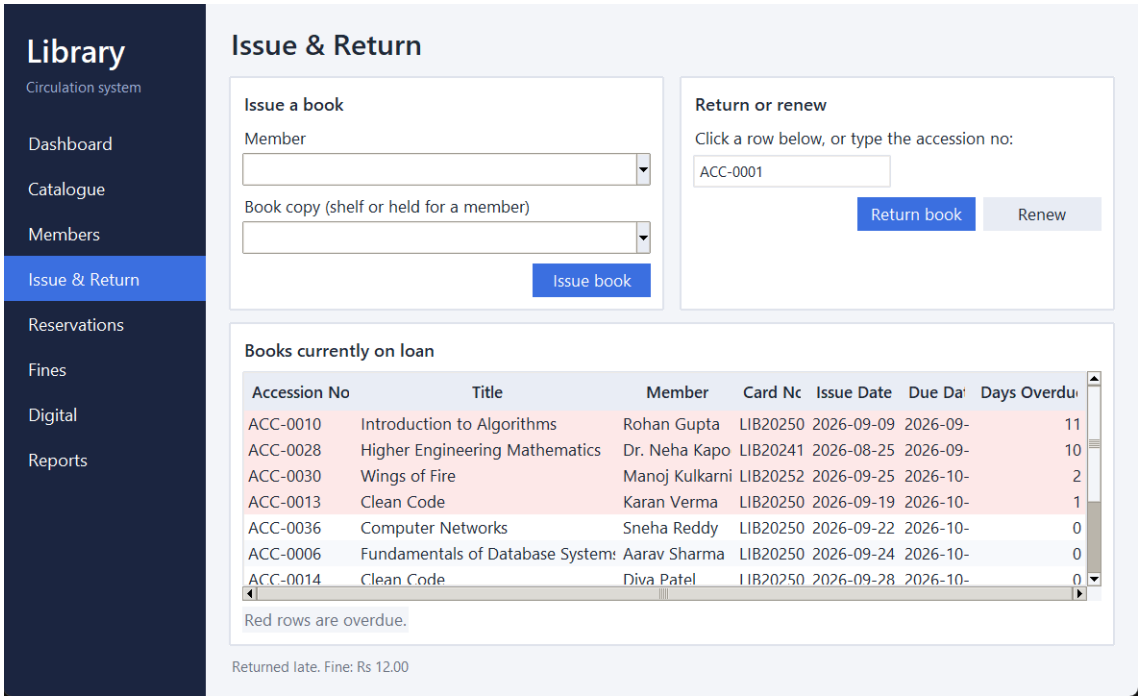


Figure 6. Returning an overdue book creates a fine

Figure 7 shows the queue. READY means a copy is being held for that member.

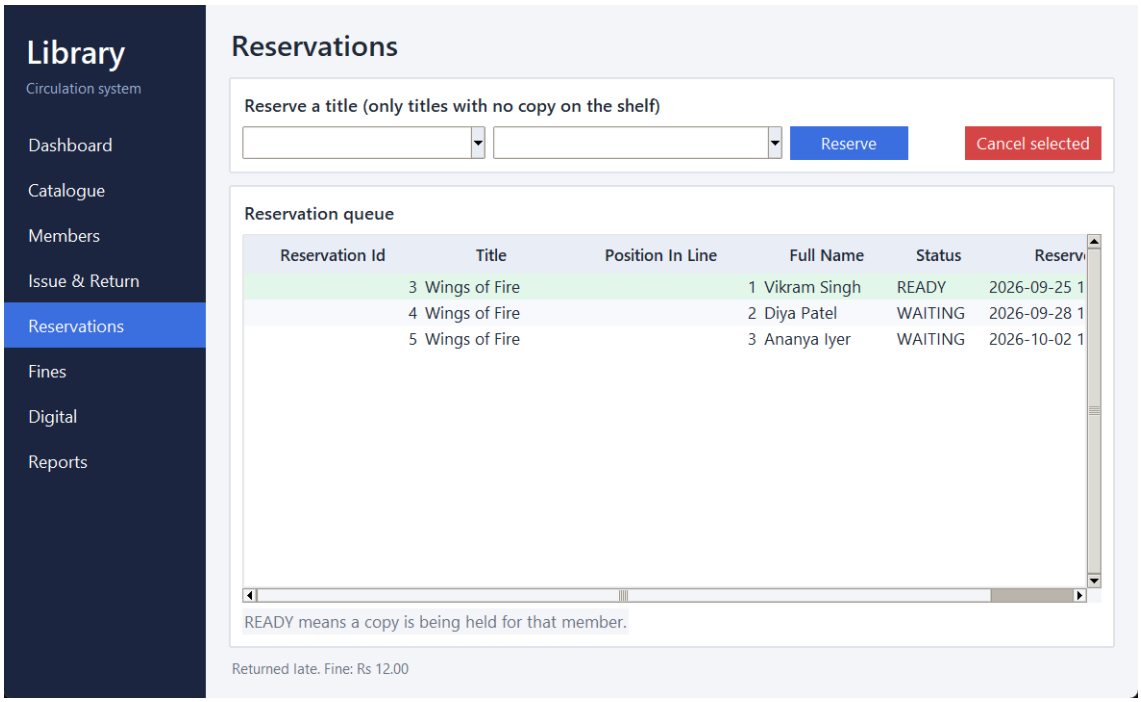


Figure 7. Reservation queue for Wings of Fire

Figure 8 lists unpaid fines. Clicking a row fills in the balance for payment.

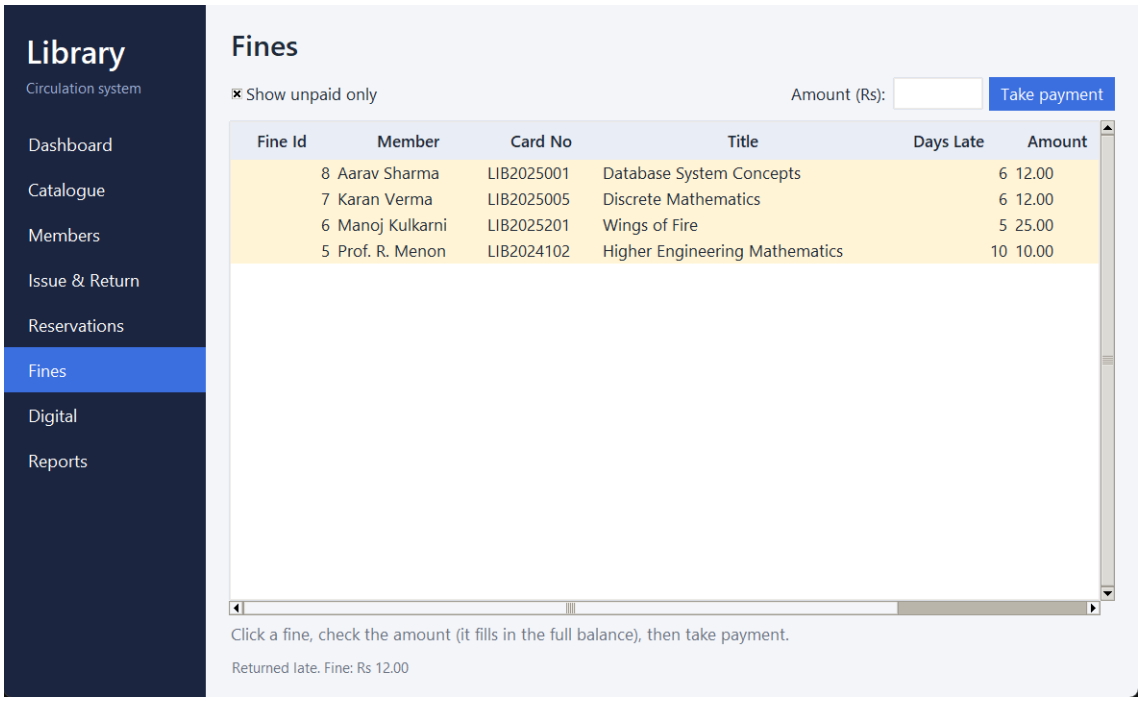


Figure 8. Unpaid fines, including the fine created by the return

Figure 9 is the overdue report with fine so far.

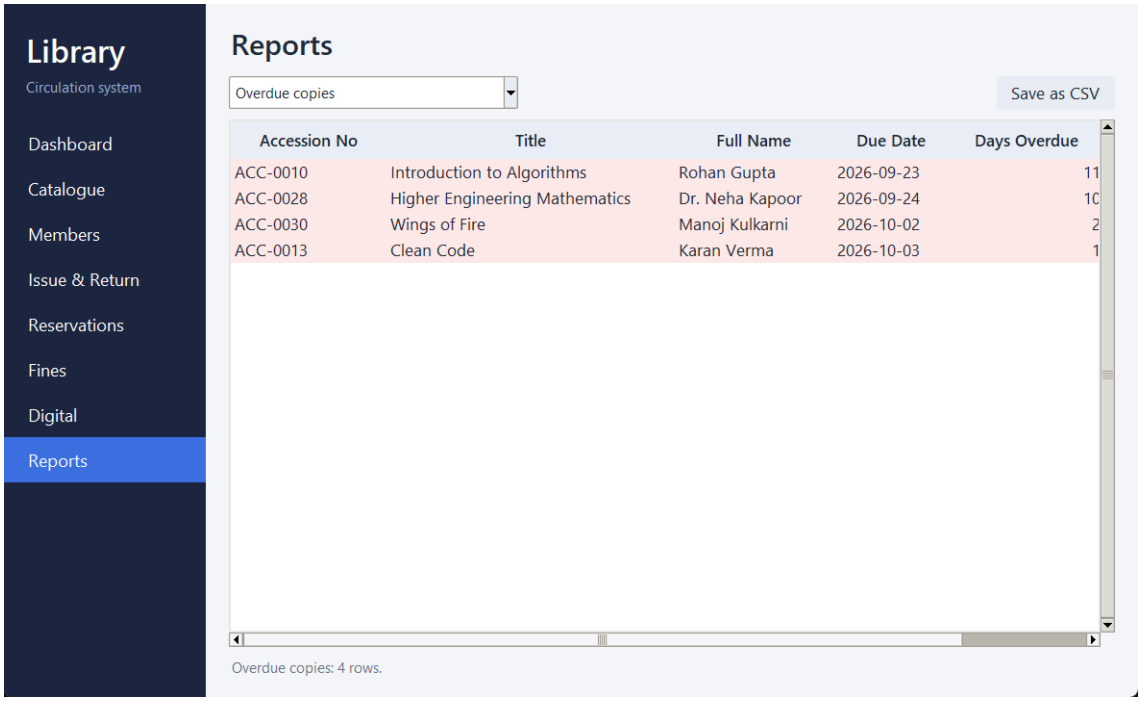


Figure 9. Report: overdue copies with fine so far

Figure 10 shows fines raised, collected and still pending for each month.

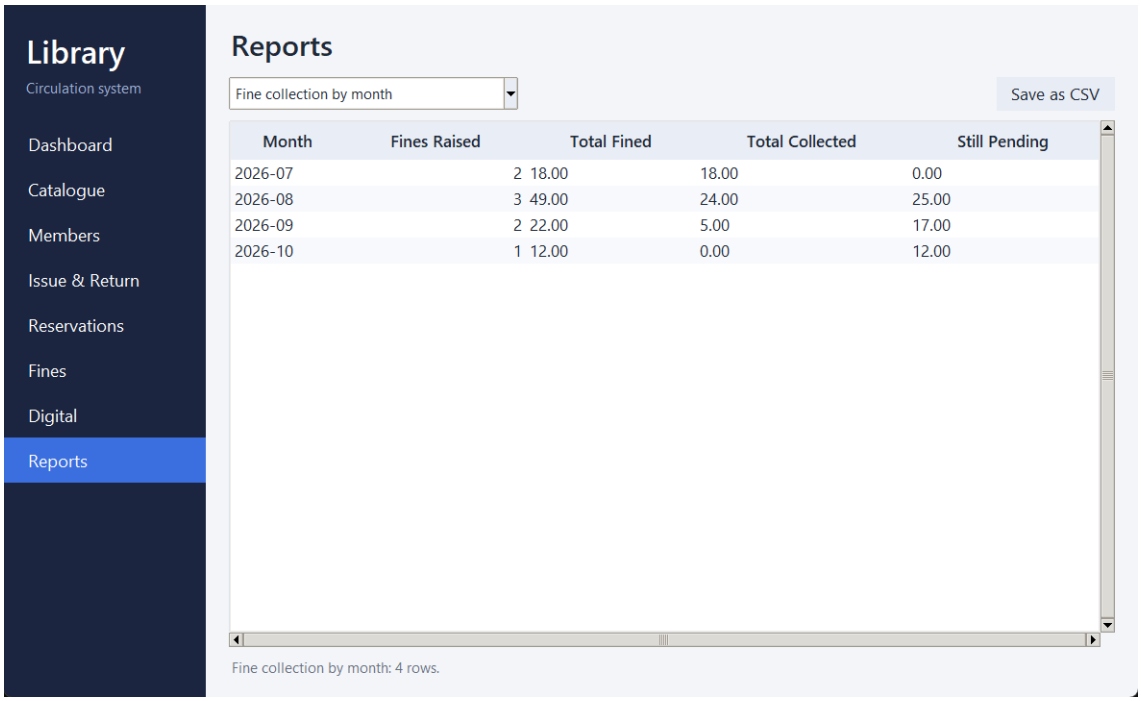


Figure 10. Report: fine collection by month

Figure 11 shows digital resource usage and the form for recording access.

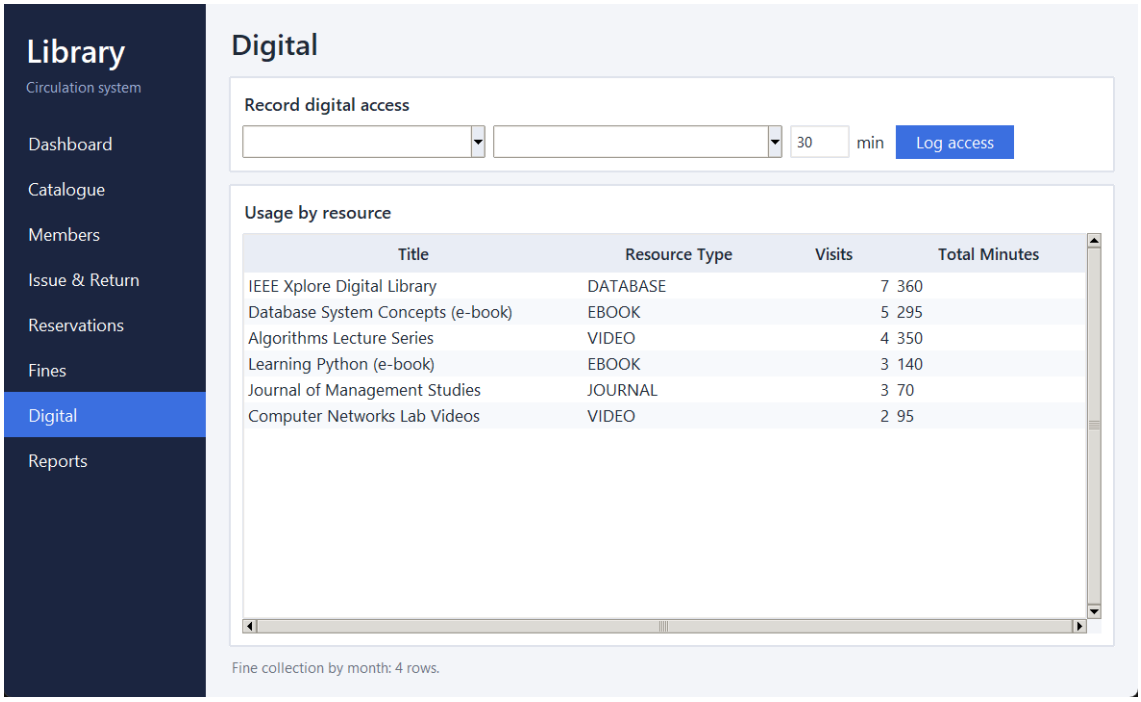


Figure 11. Digital resources and usage

3.8 Testing and Results

Table 15 lists the test cases. The expected result was written before the test was run, and the actual result is the real message or value the program produced on a freshly loaded database. The same checks, and more, are repeated in the automated test file tests/test_library.py.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Issue (positive)	Issue ACC-0002 to student LIB2025007	Loan created; due date = today + 14 days	Issued; due date 2026-10-18 (14 days). Copy became ISSUED	**Pass**
TC-02	Book limit (negative)	Issue a 4th book to the same student	Refused with limit message; nothing saved	“Limit reached: Student members can hold 3 books.” Copy ACC-0005 stayed AVAILABLE	**Pass**
TC-03	Copy already out (negative)	Issue ACC-0001, which is already on loan	Refused	“This copy is not available (status: ISSUED).”	**Pass**
TC-04	Expired member (negative)	Issue a copy to expired member LIB2023015	Refused	“Membership is expired.”	**Pass**
TC-05	Late return (positive)	Return ACC-0001, 6 days overdue, student rate Rs 2	Fine of Rs 12	Fine Rs 12.00 created for 6 days late	**Pass**
TC-06	On-time return	Return ACC-0002 the same day	No fine; copy back on shelf	Fine 0.00; copy AVAILABLE; no row added to fine	**Pass**
TC-07	Return not on loan (negative)	Return ACC-0008, which is on the shelf	Refused	“That copy is not out on loan.”	**Pass**
TC-08	Reserve while copy on shelf (negative)	Reserve Database System Concepts (copies available)	Refused	“A copy is on the shelf. Issue it directly instead of reserving.”	**Pass**
TC-09	Queue promotion	Issue last copy of Clean Code, reserve, then return another copy	Returned copy held for first in line	Reservation became READY with copy 13; copy status ON_HOLD	**Pass**
TC-10	Held copy for another member (negative)	Another member tries to issue held copy ACC-0029	Refused	“This copy is kept aside for another member.”	**Pass**
TC-11	Duplicate ISBN (negative)	Add a book with an ISBN already in the catalogue	Refused	“This ISBN is already in the catalogue.”	**Pass**
TC-12	Invalid ISBN (negative)	Add a book with ISBN 12345	Refused	“ISBN must have exactly 13 digits.”	**Pass**
TC-13	Invalid e-mail (negative)	Register member with e-mail “not-an-email”	Refused	“Email address is not valid.”	**Pass**
TC-14	Fine payment	Pay Rs 100 on a Rs 25 fine, then pay Rs 10	First refused; second accepted with balance Rs 15	“Amount is more than the balance of Rs 25.00.” then balance Rs 15.00	**Pass**

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-15	Expired licence (negative)	Log use of a resource whose licence ended	Refused	“The licence for this resource has ended.”	**Pass**
TC-16	Database rule: one active issue per copy	Insert a second active issue for copy 10 directly in SQL	MySQL rejects it	Error 1062: Duplicate entry '10' for key 'issue.active_copy_id'	**Pass**
TC-17	Database rule: negative fine	Insert a fine of -5 directly in SQL	MySQL rejects it	Error 3819: Check constraint 'fine_chk_2' is violated	**Pass**

All 17 test cases behaved as expected. Forty automated unit tests also pass on MySQL 8.0 (the full set covers input checks, catalogue, issue, return, renewal, reservation, fines, digital access and reports).

The tests show that the application uses the designed database and that the important rules are enforced twice: once in the program, with a clear message for the user, and once in the database. TC16 and TC17 bypass the program and try to break the database directly, and MySQL rejects both. The failed fourth issue in TC02 left the copy untouched, which shows the rollback works.

Two defects were found and fixed during testing. The search screen counted copies twice for titles with two authors, and a failed CHECK constraint reached the user as a raw database error. The screens themselves were checked by hand and by building every page against the database, not by automated clicking.

CHAPTER 4
CONCLUSION

This project set out to replace the paper registers of a library with a reliable database. An ER model with fifteen entities was designed and converted into a relational schema in Third Normal Form. The schema was built in MySQL with keys and constraints that enforce rules such as unique ISBN and accession numbers, one active issue per copy, valid dates, queue order and non-negative fines. Realistic sample data, twenty SQL queries and six views were prepared.

On top of the database, a Python desktop application covers the daily work of a library: catalogue maintenance, member registration, issue, return, renewal, reservation, fine payment, search and reports. The most useful lesson was that rules are safest when the database enforces them, because the program can then be changed without risking bad data. Working with transactions also showed why a multi-step action such as a return, which closes a loan, adds a fine and moves a copy to the next reader, must either finish completely or not happen at all.

4.1 Limitations and Future Enhancements

- Members cannot log in. A web version with member accounts would let them search, reserve and renew by themselves.
- Fines are raised only when a book is returned. A daily job could update the amount for books still out and send reminders.
- A reserved copy stays on hold with no time limit. A real library would expire the hold after a few days.
- Lost-book charges, fine waivers and stock purchase are not handled.
- The program connects with one database user. Separate users with limited rights for librarian and admin would improve security.
- The screens are not covered by automated GUI tests.

REFERENCES

[1] E. F. Codd, “A relational model of data for large shared data banks,” Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[2] P. P.-S. Chen, “The entity-relationship model—toward a unified view of data,” ACM Trans. Database Syst., vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[5] Oracle Corporation, MySQL 8.0 Reference Manual. Accessed: Oct. 4, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python 3 Documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://docs.python.org/3/library/tkinter.html

[7] Oracle Corporation, MySQL Connector/Python Developer Guide. Accessed: Oct. 4, 2026. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete database script is submitted as four files in the sql folder of the project:

- sql/01_schema.sql: DROP/CREATE DATABASE, all 15 tables, constraints and indexes
- sql/02_sample_data.sql: Sample data for every table
- sql/03_views.sql: Six views used by reports and screens
- sql/04_queries.sql: Twenty queries: joins, aggregates, nested queries, views, EXPLAIN

Run them in the order shown. The first script drops and rebuilds the database named library_db.

APPENDIX B - APPLICATION SOURCE CODE

The application source code and the automated tests are in the project folder submitted with this report:

- app/library.py: Database layer: connection, validation, transactions and all circulation rules
- app/app.py: Tkinter desktop application with eight screens
- tests/test_library.py: Forty automated tests (python -m unittest discover -s tests -v)

To run: load the SQL files, then start the program with python app/app.py from the project folder.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Two of the six views, as an example of how reports are built. The remaining views and the twenty queries are in the SQL files named in Appendix A.

```
-- Copies that are past their due date and not yet back
CREATE OR REPLACE VIEW v_overdue_copies AS
SELECT i.issue_id, cp.accession_no, b.title, m.card_no, m.full_name, m.email,
       i.issue_date, i.due_date,
       DATEDIFF(CURDATE(), i.due_date)                    AS days_overdue,
       DATEDIFF(CURDATE(), i.due_date) * mp.fine_per_day  AS fine_so_far
FROM issue i
JOIN copy cp          ON cp.copy_id = i.copy_id
JOIN edition e        ON e.edition_id = cp.edition_id
JOIN book b           ON b.book_id = e.book_id
JOIN member m         ON m.member_id = i.member_id
JOIN member_policy mp ON mp.member_type = m.member_type
WHERE i.return_date IS NULL AND i.due_date < CURDATE();
```

```
-- Waiting list, in queue order
CREATE OR REPLACE VIEW v_reservation_queue AS
SELECT r.reservation_id, b.title, e.edition_id, r.queue_no, m.full_name,
       r.status, r.reserved_on,
       ROW_NUMBER() OVER (PARTITION BY r.edition_id ORDER BY r.queue_no) AS
position_in_line
FROM reservation r
JOIN edition e ON e.edition_id = r.edition_id
JOIN book b    ON b.book_id = e.book_id
JOIN member m  ON m.member_id = r.member_id
WHERE r.status IN ('WAITING','READY');
```